FastSpeech 2 Text-to-Speech for English and Telugu

Technical report - implementation, training and variance-adaptor analysis. Datasets: Svarah (English), IndicTTS (Telugu).

Status: results for English, Telugu are not generated yet; sections marked *pending* fill in automatically when `scripts/make_report.py` is re-run after preprocessing, training and inference.

1. Implementation approach

The implementation starts from **ming024/FastSpeech2** (PyTorch), the reference implementation named in the assignment, and follows the FastSpeech 2 paper (Ren et al., ICLR 2021): a non-autoregressive Transformer that maps phonemes to mel-spectrograms with explicit duration, pitch and energy predictors (the *variance adaptor*), followed by a pretrained HiFi-GAN vocoder. One shared package, `fs2/`, holds the model and pipeline; `English/` and `Telugu/` contain the language-specific `preprocess.py`, `train.py`, `inference.py` and `config.yaml`, so the two models are trained completely independently.

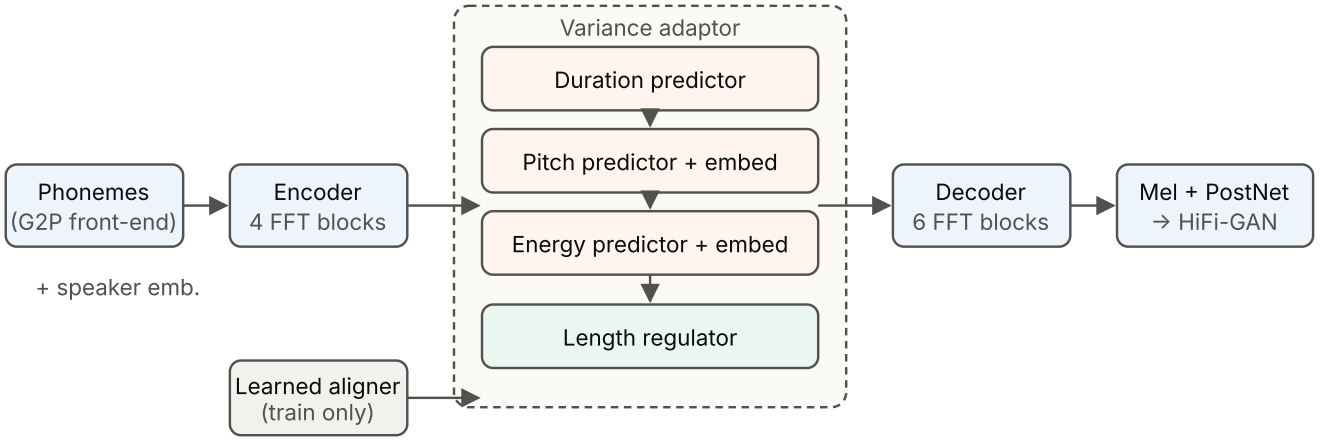


Figure 1. FastSpeech 2 as implemented. Encoder/decoder FFT blocks, PostNet and HiFi-GAN are taken unchanged from the reference repository.

Components reused from the reference implementation

- **Transformer** (`fs2/transformer/`): FFT blocks (multi-head self-attention + 1-D conv feed-forward, kernel [9, 1]), sinusoidal positions, 4 encoder / 6 decoder layers, hidden 256, 2 heads; the only edit is that vocabulary size comes from the language's symbol table.
- **PostNet**: 5 conv layers, 512 channels, residual refinement of the mel.
- **Variance predictors**: 2 x (Conv1D-ReLU-LayerNorm-Dropout) + Linear, 256 quantisation bins for pitch and energy embeddings.
- **Loss**: L1 on mel before and after PostNet, MSE on log-duration, pitch and energy.
- **Optimiser**: Adam (0.9, 0.98, eps 1e-9) with Noam warm-up (4000 steps) and step annealing; gradient clipping 1.0.

- **Vocoder**: the reference repo's pretrained HiFi-GAN *universal* generator (22.05 kHz, 80 mels, fmax 8 kHz); features are computed with the same STFT settings so no vocoder fine-tuning is needed.

Modifications for English and Telugu

Area	Reference repo	This project
Text front-end	English cleaners + g2p_en (CMUdict)	English: number/time/date normaliser written for the project, acronym spelling, g2p_en with POS-based heteronym disambiguation. Telugu: new rule-based akshara G2P (inherent vowel, virama, matras, homorganic anusvara), Telugu number words incl. ordinals (12వ) and times.
Symbols	one global ARPAbet + pinyin table	per-language symbol table saved with every checkpoint (72 English / 52 Telugu symbols incl. pad, sp, spn).
Durations	Montreal Forced Aligner TextGrids	default: learned aligner trained jointly (beta-binomial prior + forward-sum CTC loss + MAS + bin loss), because MFA has no pretrained Telugu model. MFA remains available (scripts/run_mfa.sh, alignment.method: mfa).
Pitch / energy	phoneme-level averages written to disk	frame-level F0 / energy stored; phoneme averages computed on the fly from whichever durations are used. Bins from robust quantiles of training statistics.
Variance adaptor	-	energy predictor now uses energy control (reference passes the pitch control - bug); vectorised length regulator; minimum one frame per phoneme at inference.
Data	LJSpeech / AISHELL3 / LibriTTS loaders	Svarah and IndicTTS-Telugu loaders (Hugging Face, manifests, Festival txt.done.data), quality filtering and speaker-stratified, text-disjoint 5 h / 1 h split.
Training	separate preprocess/model/train YAMLs	single config.yaml per language, best-validation checkpointing, CSV logs, automatic loss plots, resume, optional fp16 and early stopping.
Inference	synthesize.py	batch synthesis of the test sentences with duration/pitch/energy plots and numeric JSON per sentence; checkpoint is self-contained (symbols, speakers, statistics).

2. Datasets, subsampling and preprocessing

English - Svarah (AI4Bharat): 9.6 h of Indian-accented English from 117 speakers across 65 districts, read and extempore speech, released as a single split. No speaker id is published, so a stable pseudo-speaker id is derived from each utterance's speaker metadata (gender, age group, native language, state, district, qualification, job, domain). **Telugu - IndicTTS** (IIT Madras): studio recordings of one male (4.47 h) and one female (4.27 h) speaker at 48 kHz. Because 6 h cannot come from a single Telugu speaker, both speakers are used with a speaker embedding.

Selection strategy (both languages, `fs2/selection.py`, seed 1234). (1) keep 1-11 s utterances with a non-empty transcript and no annotation tags; (2) drop the 1% slowest and 1% fastest utterances by characters per second - this removes truncated audio and transcripts that do not match; (3) group utterances with identical normalised text and always place a group in one split, so no sentence occurs in both train and validation (IndicTTS speakers share prompts); (4) give every speaker a share of the 1 h validation and 5 h training budgets proportional to their share of the filtered audio, and fill validation first, then training, visiting groups in seeded random order. Assertions in the code guarantee zero utterance and zero text overlap.

Preprocessing. Resample to 22.05 kHz mono, trim leading/trailing silence (30 dB English, 35 dB Telugu), peak-normalise; text normalisation and G2P (Section 1); 80-bin log-mel (n_fft 1024, hop 256, fmax 8 kHz); F0 with WORLD DIO + StoneMask, unvoiced frames linearly interpolated; frame energy = L2 norm of the STFT magnitude; utterances longer than 1000 frames or with fewer frames than phonemes are discarded; pitch/energy are z-normalised with training-set statistics.

Language	Corpus	After filters	Speakers	Train	Validation	After feature checks
English	pending					
Telugu	pending					

Training configuration

Model	Data	Durations	Batch	Steps	Warm-up	LR anneal at	F0 search	Speaker emb.
English	Svarah	learned	16	60000	4000	30000, 40000, 50000	60-500 Hz	yes
Telugu	IndicTTS-Telugu	learned	16	60000	4000	30000, 40000, 50000	60-600 Hz	yes

Model: 35.6 M parameters (incl. 0.5 M aligner). Peak learning rate 9.9e-4 at step 4000, x0.3 at each anneal step. The checkpoint with the lowest validation mel + PostNet L1 is used for inference.

3. Training and validation loss

English

Figure pending: `Results/English/training_loss.png` *not generated yet.*

Figure pending: `Results/English/validation_loss.png` *not generated yet.*

Pending: run training to generate the loss logs.

Telugu

Figure pending: `Results/Telugu/training_loss.png` *not generated yet.*

Figure pending: `Results/Telugu/validation_loss.png` *not generated yet.*

Pending: run training to generate the loss logs.

4. Generated speech - observations per test sentence

Audio files are in `Results/<Language>/test_generated_audios/`. Measured quantities come from the model's own predictions; listening notes (pronunciation, naturalness, artefacts) are the author's, recorded in `Report/observations_<Language>.md`.

English

Pending: run inference.

Telugu

Pending: run inference.

5. Variance-adaptor analysis (duration, F0, energy)

For each test sentence the plots show the *predicted* values: duration in frames per phoneme (1 frame = 11.6 ms; grey bars are pauses), F0 in Hz after de-normalisation, and frame energy. The top panel of the pitch and energy figures is the frame-level contour the decoder actually receives after length regulation; the lower panel is the per-phoneme predictor output.

English

Pending: run inference.

Telugu

Pending: run inference.

6. English vs Telugu - comparison, limitations, improvements

Model	Speakers	Best val mel+PostNet L1	Val duration	Val pitch	Phonemes/s (tests)	Mean F0 range / sentence
English	-	pending	-	-	-	-
Telugu	-	pending	-	-	-	-

Data. Telugu has two studio speakers reading prepared text; Svarah spreads 6 h over ~117 speakers with varied microphones, accents and partly extempore speech, so each English speaker contributes only a few minutes. Expect the Telugu model to reach a lower mel loss and sound cleaner and more consistent, while the English model averages over speakers (flatter prosody, occasional speaker drift) - a property of the data rather than the architecture.

Front-end. Telugu spelling is close to phonemic, so the rule-based G2P is reliable for native words; its weak points are loan words written in Telugu script (T1: schwa deletion and English vowel qualities are not modelled) and the homograph పట్టు (T2), which no grapheme-level model can resolve. English depends on CMUdict plus a neural G2P: it handles the heteronym pair (E2) through POS tagging, but must guess OOV words (E1 is in CMUdict; 'rhyth', 'andaroo' are not) and is defeated by the malformed token in E3, while the same error in Telugu (T3) is harmless because spaces are not pronounced.

Numbers. Both front-ends expand the date/time sentences (E4, T4) deterministically; Telugu needs inflected forms (వేల vs వేలు, the ordinal పన్నెండవ), which a naive digit-to-word mapping gets wrong.

Limitations. ~5 h per language is about a fifth of LJSpeech, so expect some mumbling on long or rare words, limited prosodic variety and occasional duration errors; the universal HiFi-GAN is not fine-tuned to these voices (slight buzz/metallic artefacts); learned alignments can be imprecise on very short phonemes; global pitch normalisation is shared between male and female voices.

Improvements. (1) fine-tune HiFi-GAN on ground-truth-aligned mels of each corpus; (2) per-speaker pitch normalisation; (3) a Telugu lexicon for English loan words and a context-aware homograph model; (4) input text validation (merge stray fragments, E3); (5) pretrain on a larger multilingual

Indic/English set and fine-tune on the 5 h subsets; (6) MFA durations for English to compare with the learned aligner; (7) for Svarah, restrict training to the cleanest read-speech speakers or add a reference encoder for speaker/style.

7. Reproducibility

Python 3.9-3.13, PyTorch 2.x (CUDA recommended; CPU works for inference). Dependencies (`requirements.txt`):

```
# core (tested with Python 3.13, torch 2.15 CPU; any torch >= 2.3 with CUDA is recommended for
training)
torch>=2.3
numpy>=1.23
scipy>=1.10
librosa>=0.10
soundfile>=0.12
pyworld>=0.3.4
numba>=0.57
g2p_en>=2.1
nltk>=3.8
PyYAML>=6.0
tqdm>=4.65
matplotlib>=3.7
# dataset download (Svarah / IndicTTS from Hugging Face)
datasets>=2.18
huggingface_hub>=0.22
# optional: Montreal Forced Aligner TextGrids (alignment.method: mfa)
tgt>=1.4
# optional: build Report/Technical_Report.pdf  (then: playwright install chromium)
playwright>=1.40
```

```
pip install -r requirements.txt
python -c "import nltk; [nltk.download(p) for p in ('averaged_perceptron_tagger_eng','cmudict')]"
huggingface-cli login                     # Svarah requires accepting its terms on Hugging Face

python scripts/smoke_test.py              # 5-minute end-to-end check on synthetic data

python English/preprocess.py              # select 5 h / 1 h, extract features
python English/train.py                   # -> English/checkpoints/best.pth.tar,
Results/English/*loss.png
python English/inference.py               # -> Results/English/test_generated_audios,
variance_adaptor_plots

python Telugu/preprocess.py               # or: --source festival --corpus_path
/path/IndicTTS/Telugu
python Telugu/train.py
python Telugu/inference.py

python scripts/make_report.py             # rebuild this report
```

All hyper-parameters live in `English/config.yaml` and `Telugu/config.yaml` ; seeds are fixed (1234) for data selection, shuffling and initialisation. Checkpoints embed the symbol table, speaker table and normalisation statistics, so `inference.py` needs only the checkpoint and the config.

References

[1] Y. Ren et al., "FastSpeech 2: Fast and High-Quality End-to-End Text to Speech", ICLR 2021. [2] C.-M. Chien, ming024/FastSpeech2, GitHub. [3] J. Kong et al., "HiFi-GAN", NeurIPS 2020. [4] R. Badlani et al., "One TTS Alignment to Rule Them All", ICASSP 2022. [5] AI4Bharat, Svarah: Indian-accented English benchmark, 2023. [6] IIT Madras, Indic TTS database.